KRONOS PA Scalp Research v2 | Briefing previsionnel

Pine Script v6 selectionnable, avec numeros de ligne. Le panneau inclut contexte d'une unite superieure, score PA, cible de scenario et invalidation.

```
   1  //@version=6
   2  strategy("KRONOS — Price Action Scalp Research v2", shorttitle="KRONOS PA R2", overlay=true, pyramiding=0, initial_capital=1000, currency=currency.USD, default_qty_type=strategy.fixed,
default_qty_value=1, commission_type=strategy.commission.percent, commission_value=0.0, slippage=1, process_orders_on_close=false, max_labels_count=100, max_boxes_count=10)
   3
   4  // Research implementation of the two rule-based PA setups tested locally.
   5  // It is not a profitability claim. Use M1/M5 and set realistic Strategy Tester
   6  // commission, spread and slippage for the exact symbol/feed before evaluating.
   7
   8  string G_SETUP = "01 · Setup Price Action"
   9  string G_RISK = "02 · Risque et couts"
  10  string G_EXIT = "03 · Sorties scalping"
  11  string G_VIEW = "04 · Graphique et alertes"
  12
  13  string setupMode = input.string("Cassure + retest", "Setup a trader", options=["Cassure + retest", "Rejet confirme"], group=G_SETUP, tooltip="Deux variantes testees separement. Aucune n'a
encore montre un avantage robuste apres frais.")
  14  int srLookback = input.int(30, "Fenetre support/resistance (bougies)", minval=5, maxval=300, group=G_SETUP)
  15  int forecastHorizonBars = input.int(5, "Horizon du scenario (bougies)", minval=1, maxval=30, group=G_SETUP, tooltip="Score PA: structure locale +/-2, cassure/retest +/-2, contexte superieur
+/-2, position dans le range +/-1, momentum sur cet horizon +/-1. Score de confluence, pas une probabilite.")
  16  string contextTimeframe = input.timeframe("15", "Unite superieure de contexte", group=G_SETUP, tooltip="Utilise la derniere bougie cloturee de cette unite. Sur un graphique M15, selectionne H1
pour conserver un vrai contexte superieur.")
  17  int pivotConfirmBars = input.int(3, "Confirmation des swings (bougies)", minval=1, maxval=10, group=G_SETUP, tooltip="Un swing n'est retenu qu'apres confirmation de ces bougies a droite.")
  18  int retestBars = input.int(8, "Expiration du retest (bougies)", minval=1, maxval=50, group=G_SETUP)
  19  float toleranceRange = input.float(0.20, "Tolerance retest/rejet (x range moyen)", minval=0.01, maxval=1.0, step=0.01, group=G_SETUP)
  20  float breakoutBufferPct = input.float(0.02, "Marge de cassure (%)", minval=0.0, maxval=1.0, step=0.01, group=G_SETUP)
  21  int rangePeriod = input.int(14, "Moyenne du range high-low", minval=2, maxval=200, group=G_SETUP)
  22
  23  float riskPct = input.float(0.50, "Risque par trade (% equity)", minval=0.05, maxval=2.0, step=0.05, group=G_RISK, tooltip="0,50% correspond au calcul illustratif du backtest. Cela ne garantit
pas la perte maximale en cas de gap/slippage.")
  24  float stopBufferRange = input.float(0.10, "Marge derriere l'invalidation (x range moyen)", minval=0.01, maxval=1.0, step=0.01, group=G_RISK)
  25  float maxStopRange = input.float(3.0, "Risque maximal du stop (x range moyen)", minval=0.25, maxval=10.0, step=0.25, group=G_RISK)
  26  float estimatedSpreadPct = input.float(0.0039, "Spread estime (%)", minval=0.0, maxval=1.0, step=0.0001, group=G_RISK, tooltip="Sert uniquement au filtre de cout du setup; ce spread n'est pas
debite du P&L Strategy Tester. Modelise les couts dans Proprietes de la strategie.")
  27  float commissionPctPerSide = input.float(0.0, "Commission par cote (%)", minval=0.0, maxval=1.0, step=0.001, group=G_RISK, tooltip="Pour le Strategy Tester, regle aussi la commission reelle
dans Proprietes de la strategie.")
  28  int slippageTicksPerSide = input.int(1, "Slippage estime (ticks par cote)", minval=0, maxval=100, group=G_RISK, tooltip="Sert au filtre de cout; le slippage du Strategy Tester se regle aussi
dans Proprietes de la strategie.")
  29  float maxCostR = input.float(0.15, "Cout aller-retour maximal (R)", minval=0.0, maxval=1.0, step=0.01, group=G_RISK)
  30
  31  float tp1R = input.float(1.0, "TP1 (R)", minval=0.25, maxval=10.0, step=0.05, group=G_EXIT)
  32  float tp2R = input.float(1.6, "TP2 (R)", minval=0.5, maxval=20.0, step=0.05, group=G_EXIT)
  33  float tp1QtyPct = input.float(50.0, "Position fermee a TP1 (%)", minval=1.0, maxval=99.0, step=1.0, group=G_EXIT)
  34  int maxHoldMinutes = input.int(30, "Duree maximale (minutes)", minval=5, maxval=1440, group=G_EXIT)
  35
  36  bool showLevels = input.bool(true, "Afficher support/resistance et niveau arme", group=G_VIEW)
  37  bool showForecastLevels = input.bool(true, "Afficher cible et invalidation du scenario", group=G_VIEW)
  38  bool showZones = input.bool(true, "Afficher les zones colorees", group=G_VIEW, tooltip="Les bandes colorent la tolerance autour du support, de la resistance et du retest. La bande n'est pas un
signal d'entree a elle seule.")
  39  bool showSignals = input.bool(true, "Afficher les signaux confirmes", group=G_VIEW)
  40  bool showStatus = input.bool(true, "Afficher le statut", group=G_VIEW)
  41  string panelMode = input.string("Briefing pro", "Format du panneau", options=["Briefing pro", "Mobile compact"], group=G_VIEW, tooltip="Le mode compact garde le biais, le contexte, la cible,
l'invalidation et le declencheur visibles; le briefing pro ajoute le detail du score et les niveaux de gestion.")
  42  bool alertBreakoutWatch = input.bool(true, "Alerter cassure et annulation du retest", group=G_VIEW, tooltip="Envoie une alerte de surveillance apres la cassure, puis une autre si le niveau est
invalide ou si le retest expire.")
  43  bool alertForecastChanges = input.bool(true, "Alerter les changements de prevision", group=G_VIEW, tooltip="Alerte quand le biais Price Action passe entre haussier, neutre et baissier. Ce
n'est pas un signal d'entree.")
  44
  45  float avgRange = ta.sma(high - low, rangePeriod)
  46  float slowRange = ta.sma(high - low, 50)
```

```
  47  float support = ta.lowest(low[1], srLookback)
  48  float resistance = ta.highest(high[1], srLookback)
  49  float rangeMidpoint = (support + resistance) / 2.0
  50  float tolerance = math.max(avgRange * toleranceRange, syminfo.mintick * 2.0)
  51  float breakBuffer = math.max(close * breakoutBufferPct / 100.0, syminfo.mintick * 2.0)
  52  float stopBuffer = math.max(avgRange * stopBufferRange, syminfo.mintick * 2.0)
  53
  54  float candleBody = math.abs(close - open)
  55  float candleRange = math.max(high - low, close * 0.00001)
  56  float lowerWick = math.min(open, close) - low
  57  float upperWick = high - math.max(open, close)
  58  bool bullWick = close > open and lowerWick >= math.max(candleBody * 1.35, candleRange * 0.25) and close >= low + candleRange * 0.62
  59  bool bearWick = close < open and upperWick >= math.max(candleBody * 1.35, candleRange * 0.25) and close <= high - candleRange * 0.62
  60  bool bullEngulf = close[1] < open[1] and close > open and close >= open[1] and open <= close[1]
  61  bool bearEngulf = close[1] > open[1] and close < open and close <= open[1] and open >= close[1]
  62  float confirmedPivotHigh = ta.pivothigh(high, pivotConfirmBars, pivotConfirmBars)
  63  float confirmedPivotLow = ta.pivotlow(low, pivotConfirmBars, pivotConfirmBars)
  64  var float previousPivotHigh = na
  65  var float latestPivotHigh = na
  66  var float previousPivotLow = na
  67  var float latestPivotLow = na
  68  if barstate.isconfirmed and not na(confirmedPivotHigh)
  69      previousPivotHigh := latestPivotHigh
  70      latestPivotHigh := confirmedPivotHigh
  71  if barstate.isconfirmed and not na(confirmedPivotLow)
  72      previousPivotLow := latestPivotLow
  73      latestPivotLow := confirmedPivotLow
  74  bool bullishStructure = not na(previousPivotHigh) and not na(latestPivotHigh) and not na(previousPivotLow) and not na(latestPivotLow) and latestPivotHigh > previousPivotHigh and latestPivotLow
> previousPivotLow
  75  bool bearishStructure = not na(previousPivotHigh) and not na(latestPivotHigh) and not na(previousPivotLow) and not na(latestPivotLow) and latestPivotHigh < previousPivotHigh and latestPivotLow
< previousPivotLow
  76
  77  // Use only the last completed higher-timeframe candles for stable context.
  78  [contextOpen, contextHigh, contextLow, contextClose, previousContextHigh, previousContextLow, previousContextClose] = request.security(syminfo.tickerid, contextTimeframe, [open[1], high[1],
low[1], close[1], high[2], low[2], close[2]], gaps=barmerge.gaps_off, lookahead=barmerge.lookahead_on)
  79  bool contextTimeframeOk = timeframe.in_seconds(contextTimeframe) > timeframe.in_seconds()
  80  bool bullishContext = contextTimeframeOk and contextHigh > previousContextHigh and contextLow > previousContextLow and contextClose > previousContextClose and contextClose > contextOpen and
contextClose >= contextLow + (contextHigh - contextLow) * 0.65
  81  bool bearishContext = contextTimeframeOk and contextHigh < previousContextHigh and contextLow < previousContextLow and contextClose < previousContextClose and contextClose < contextOpen and
contextClose <= contextLow + (contextHigh - contextLow) * 0.35
  82
  83  // The outlook is a rule-based scenario score, not a probability of winning.
  84  // A target uses the nearest rolling S/R boundary, or a range-width projection after a break.
  85
  86  var int armedDirection = 0
  87  var float armedLevel = na
  88  var int armedBar = na
  89  bool retestLong = false
  90  bool retestShort = false
  91  bool bullBreakEvent = false
  92  bool bearBreakEvent = false
  93  bool breakoutCanceledEvent = false
  94  int canceledDirection = 0
  95  float canceledLevel = na
  96  string canceledReason = ""
  97
  98  if barstate.isconfirmed and setupMode == "Cassure + retest"
  99      if armedDirection != 0
 100          bool expired = bar_index - armedBar > retestBars
 101          bool invalidated = (armedDirection == 1 and close < armedLevel - tolerance) or (armedDirection == -1 and close > armedLevel + tolerance)
 102          if expired or invalidated
 103              breakoutCanceledEvent := true
 104              canceledDirection := armedDirection
 105              canceledLevel := armedLevel
```

```
            canceledReason := expired ? "delai du retest expire" : "niveau de cassure invalide"
            armedDirection := 0
            armedLevel := na
            armedBar := na

    bool bullBreak = armedDirection == 0 and close > resistance + breakBuffer and close[1] <= resistance + breakBuffer
    bool bearBreak = armedDirection == 0 and close < support - breakBuffer and close[1] >= support - breakBuffer
    if bullBreak
        bullBreakEvent := true
        armedDirection := 1
        armedLevel := resistance
        armedBar := bar_index
    else if bearBreak
        bearBreakEvent := true
        armedDirection := -1
        armedLevel := support
        armedBar := bar_index

    if armedDirection == 1 and bar_index > armedBar
        bool retested = low <= armedLevel + tolerance and low >= armedLevel - tolerance * 2.0
        retestLong := retested and close > armedLevel and (close > open or bullWick or bullEngulf)
    else if armedDirection == -1 and bar_index > armedBar
        bool retested = high >= armedLevel - tolerance and high <= armedLevel + tolerance * 2.0
        retestShort := retested and close < armedLevel and (close < open or bearWick or bearEngulf)

bool rejectionLong = (bullWick or bullEngulf) and low <= support + tolerance and low >= support - tolerance * 1.5 and close > support
bool rejectionShort = (bearWick or bearEngulf) and high >= resistance - tolerance and high <= resistance + tolerance * 1.5 and close < resistance
bool rawLong = setupMode == "Cassure + retest" ? retestLong : rejectionLong
bool rawShort = setupMode == "Cassure + retest" ? retestShort : rejectionShort
float signalLevel = setupMode == "Cassure + retest" ? armedLevel : rawLong ? support : resistance

if setupMode == "Cassure + retest" and (retestLong or retestShort)
    armedDirection := 0
    armedLevel := na
    armedBar := na

int structureScore = bullishStructure ? 2 : bearishStructure ? -2 : 0
int breakoutScore = (armedDirection == 1 or retestLong) ? 2 : (armedDirection == -1 or retestShort) ? -2 : 0
int rangePositionScore = close > rangeMidpoint ? 1 : close < rangeMidpoint ? -1 : 0
int horizonMomentumScore = close > close[forecastHorizonBars] ? 1 : close < close[forecastHorizonBars] ? -1 : 0
int contextScore = bullishContext ? 2 : bearishContext ? -2 : 0
int forecastScore = structureScore + breakoutScore + contextScore + rangePositionScore + horizonMomentumScore
int forecastDirection = forecastScore >= 3 ? 1 : forecastScore <= -3 ? -1 : 0
bool forecastChanged = barstate.isconfirmed and not na(forecastDirection[1]) and forecastDirection != forecastDirection[1]
float projectionRange = math.max(resistance - support, avgRange)
float forecastTarget = forecastDirection == 1 ? (close < resistance ? resistance : close + projectionRange) : forecastDirection == -1 ? (close > support ? support : close - projectionRange) :
na
float forecastInvalidation = forecastDirection == 1 ? (armedDirection == 1 and not na(armedLevel) ? armedLevel - tolerance : retestLong ? signalLevel - tolerance : support) : forecastDirection
== -1 ? (armedDirection == -1 and not na(armedLevel) ? armedLevel + tolerance : retestShort ? signalLevel + tolerance : resistance) : na

bool chartTimeframeOk = timeframe.isminutes and timeframe.multiplier <= 15
bool confirmedLong = barstate.isconfirmed and chartTimeframeOk and rawLong and not rawShort and strategy.position_size == 0
bool confirmedShort = barstate.isconfirmed and chartTimeframeOk and rawShort and not rawLong and strategy.position_size == 0
float longStop = math.min(signalLevel, low) - stopBuffer
float shortStop = math.max(signalLevel, high) + stopBuffer
float longRisk = close - longStop
float shortRisk = shortStop - close
float estimatedCost = close * (estimatedSpreadPct + 2.0 * commissionPctPerSide) / 100.0 + syminfo.mintick * slippageTicksPerSide * 2.0
bool longPlanValid = not na(avgRange) and avgRange > 0 and longRisk > 0 and longRisk <= avgRange * maxStopRange and estimatedCost / longRisk <= maxCostR
bool shortPlanValid = not na(avgRange) and avgRange > 0 and shortRisk > 0 and shortRisk <= avgRange * maxStopRange and estimatedCost / shortRisk <= maxCostR
bool enterLong = confirmedLong and longPlanValid
bool enterShort = confirmedShort and shortPlanValid

float riskCash = strategy.equity * riskPct / 100.0
```

```
float longRiskPerUnit = strategy.convert_to_account(longRisk * syminfo.pointvalue)
float shortRiskPerUnit = strategy.convert_to_account(shortRisk * syminfo.pointvalue)
float rawLongQty = longRiskPerUnit > 0 ? riskCash / longRiskPerUnit : na
float rawShortQty = shortRiskPerUnit > 0 ? riskCash / shortRiskPerUnit : na
float qtyStep = syminfo.mincontract
float longQty = not na(rawLongQty) and qtyStep > 0 ? math.floor(rawLongQty / qtyStep) * qtyStep : na
float shortQty = not na(rawShortQty) and qtyStep > 0 ? math.floor(rawShortQty / qtyStep) * qtyStep : na
bool longQtyValid = not na(longQty) and qtyStep > 0 and longQty >= qtyStep * 2.0
bool shortQtyValid = not na(shortQty) and qtyStep > 0 and shortQty >= qtyStep * 2.0

var float activeStop = na
var int activeMaxHoldBars = na
var int activeTp1Ticks = na
var int activeTp2Ticks = na
int chartMinutes = timeframe.isminutes ? timeframe.multiplier : 0
int maxHoldBars = chartMinutes > 0 ? math.max(1, int(math.ceil(maxHoldMinutes / chartMinutes))) : na
bool placeLong = enterLong and longQtyValid and chartTimeframeOk
bool placeShort = enterShort and shortQtyValid and chartTimeframeOk

if alertForecastChanges and chartTimeframeOk and forecastChanged
    string forecastAlertSide = forecastDirection == 1 ? "HAUSSIERE" : forecastDirection == -1 ? "BAISSIERE" : "NEUTRE"
    string forecastHorizonMessage = str.tostring(forecastHorizonBars * timeframe.multiplier) + " min environ"
    string forecastAlertDetails = forecastDirection == 1 ? "scenario vers " + str.tostring(forecastTarget, format.mintick) + "; invalide si cloture sous " + str.tostring(forecastInvalidation,
format.mintick) : forecastDirection == -1 ? "scenario vers " + str.tostring(forecastTarget, format.mintick) + "; invalide si cloture au-dessus de " + str.tostring(forecastInvalidation,
format.mintick) : "range; seuils de sortie: au-dessus de " + str.tostring(resistance, format.mintick) + " ou sous " + str.tostring(support, format.mintick)
    alert("KRONOS PA | PREVISION " + forecastAlertSide + " | " + syminfo.ticker + " " + timeframe.period + " | score PA=" + str.tostring(forecastScore) + "/8 | contexte=" + (bullishContext ?
"haussier" : bearishContext ? "baissier" : "mixte") + " | horizon=" + forecastHorizonMessage + " | " + forecastAlertDetails, alert.freq_once_per_bar_close)
if alertBreakoutWatch and bullBreakEvent
    alert("KRONOS PA | CASSURE HAUSSIERE A SURVEILLER | " + syminfo.ticker + " " + timeframe.period + " | niveau=" + str.tostring(armedLevel, format.mintick) + " | attendre le retest et un
rejet; ne pas entrer sur la cassure seule", alert.freq_once_per_bar_close)
if alertBreakoutWatch and bearBreakEvent
    alert("KRONOS PA | CASSURE BAISSIERE A SURVEILLER | " + syminfo.ticker + " " + timeframe.period + " | niveau=" + str.tostring(armedLevel, format.mintick) + " | attendre le retest et un
rejet; ne pas entrer sur la cassure seule", alert.freq_once_per_bar_close)
if alertBreakoutWatch and breakoutCanceledEvent
    string canceledSide = canceledDirection == 1 ? "HAUSSIERE" : "BAISSIERE"
    alert("KRONOS PA | CASSURE " + canceledSide + " ANNULEE | " + syminfo.ticker + " " + timeframe.period + " | niveau=" + str.tostring(canceledLevel, format.mintick) + " | raison=" +
canceledReason, alert.freq_once_per_bar_close)
if placeLong
    activeStop := longStop
    activeMaxHoldBars := maxHoldBars
    activeTp1Ticks := math.max(1, int(math.round(longRisk * tp1R / syminfo.mintick)))
    activeTp2Ticks := math.max(1, int(math.round(longRisk * tp2R / syminfo.mintick)))
    string fillMessage = "KRONOS PA scalp BUY " + syminfo.ticker + " " + timeframe.period + " | setup=" + setupMode + " | signalClose=" + str.tostring(close, format.mintick) + " | SL=" +
str.tostring(longStop, format.mintick) + " | TP1=" + str.tostring(close + longRisk * tp1R, format.mintick) + " | TP2=" + str.tostring(close + longRisk * tp2R, format.mintick) + " | risk=" +
str.tostring(riskPct) + "% | fill=next available tick"
    strategy.entry("Long", strategy.long, qty=longQty, alert_message=fillMessage)
    strategy.exit("Long TP1", from_entry="Long", stop=activeStop, profit=activeTp1Ticks, qty_percent=tp1QtyPct)
    strategy.exit("Long TP2", from_entry="Long", stop=activeStop, profit=activeTp2Ticks, qty_percent=100.0 - tp1QtyPct)
if placeShort
    activeStop := shortStop
    activeMaxHoldBars := maxHoldBars
    activeTp1Ticks := math.max(1, int(math.round(shortRisk * tp1R / syminfo.mintick)))
    activeTp2Ticks := math.max(1, int(math.round(shortRisk * tp2R / syminfo.mintick)))
    string fillMessage = "KRONOS PA scalp SELL " + syminfo.ticker + " " + timeframe.period + " | setup=" + setupMode + " | signalClose=" + str.tostring(close, format.mintick) + " | SL=" +
str.tostring(shortStop, format.mintick) + " | TP1=" + str.tostring(close - shortRisk * tp1R, format.mintick) + " | TP2=" + str.tostring(close - shortRisk * tp2R, format.mintick) + " | risk=" +
str.tostring(riskPct) + "% | fill=next available tick"
    strategy.entry("Short", strategy.short, qty=shortQty, alert_message=fillMessage)
    strategy.exit("Short TP1", from_entry="Short", stop=activeStop, profit=activeTp1Ticks, qty_percent=tp1QtyPct)
    strategy.exit("Short TP2", from_entry="Short", stop=activeStop, profit=activeTp2Ticks, qty_percent=100.0 - tp1QtyPct)

int barsInTrade = strategy.opentrades > 0 ? bar_index - strategy.opentrades.entry_bar_index(0) : 0
bool timeExit = strategy.position_size != 0 and not na(activeMaxHoldBars) and barsInTrade >= activeMaxHoldBars
if timeExit
    strategy.cancel("Long TP1")
```

```
    strategy.cancel("Long TP2")
    strategy.cancel("Short TP1")
    strategy.cancel("Short TP2")
    strategy.close_all(comment="Time stop", immediately=true)
if strategy.position_size == 0 and strategy.position_size[1] != 0
    activeStop := na
    activeMaxHoldBars := na
    activeTp1Ticks := na
    activeTp2Ticks := na

plot(showLevels ? support : na, "Support 30 barres", color=color.new(color.lime, 10), linewidth=2, style=plot.style_linebr)
plot(showLevels ? resistance : na, "Resistance 30 barres", color=color.new(color.red, 10), linewidth=2, style=plot.style_linebr)
plot(showLevels and armedDirection != 0 ? armedLevel : na, "Cassure en attente de retest", color=color.orange, linewidth=2, style=plot.style_linebr)
plot(showLevels and strategy.position_size != 0 ? activeStop : na, "Stop structurel actif", color=color.red, linewidth=2, style=plot.style_linebr)
float longTp1Price = strategy.position_size > 0 and not na(activeTp1Ticks) ? strategy.position_avg_price + activeTp1Ticks * syminfo.mintick : na
float longTp2Price = strategy.position_size > 0 and not na(activeTp2Ticks) ? strategy.position_avg_price + activeTp2Ticks * syminfo.mintick : na
float shortTp1Price = strategy.position_size < 0 and not na(activeTp1Ticks) ? strategy.position_avg_price - activeTp1Ticks * syminfo.mintick : na
float shortTp2Price = strategy.position_size < 0 and not na(activeTp2Ticks) ? strategy.position_avg_price - activeTp2Ticks * syminfo.mintick : na
plot(showLevels ? longTp1Price : na, "Objectif TP1 long", color=color.new(color.aqua, 10), linewidth=2, style=plot.style_linebr)
plot(showLevels ? longTp2Price : na, "Objectif TP2 long", color=color.new(color.teal, 10), linewidth=2, style=plot.style_linebr)
plot(showLevels ? shortTp1Price : na, "Objectif TP1 short", color=color.new(color.aqua, 10), linewidth=2, style=plot.style_linebr)
plot(showLevels ? shortTp2Price : na, "Objectif TP2 short", color=color.new(color.teal, 10), linewidth=2, style=plot.style_linebr)
plot(showForecastLevels and forecastDirection != 0 ? forecastTarget : na, "Scenario PA - cible indicative", color=forecastDirection == 1 ? color.new(color.lime, 0) : color.new(color.red, 0),
linewidth=2, style=plot.style_linebr)
plot(showForecastLevels and forecastDirection != 0 ? forecastInvalidation : na, "Scenario PA - invalidation", color=color.new(color.gray, 0), linewidth=2, style=plot.style_linebr)
plotshape(showSignals and bullBreakEvent, title="Cassure haussiere: attendre le retest", style=shape.labelup, location=location.belowbar, color=color.new(color.orange, 0),
text="CASSURE\nATTENDRE RETEST", textcolor=color.black, size=size.small)
plotshape(showSignals and bearBreakEvent, title="Cassure baissiere: attendre le retest", style=shape.labeldown, location=location.abovebar, color=color.new(color.orange, 0),
text="CASSURE\nATTENDRE RETEST", textcolor=color.black, size=size.small)
plotshape(showSignals and placeLong, title="Entree achat confirmee", style=shape.labelup, location=location.belowbar, color=color.new(color.lime, 0), text="ACHAT PA\nCONFIRME",
textcolor=color.black, size=size.small)
plotshape(showSignals and placeShort, title="Entree vente confirmee", style=shape.labeldown, location=location.abovebar, color=color.new(color.red, 0), text="VENTE PA\nCONFIRMEE",
textcolor=color.white, size=size.small)
plotshape(showSignals and (confirmedLong and (not longPlanValid or not longQtyValid) or confirmedShort and (not shortPlanValid or not shortQtyValid)), title="Setup bloque par risque, cout ou
taille", style=shape.labeldown, location=location.abovebar, color=color.orange, text="SETUP\nBLOQUE", textcolor=color.black, size=size.tiny)

var box supportZone = na
var box resistanceZone = na
var box retestZone = na
if barstate.islast
    if showLevels and showZones and not na(support)
        if na(supportZone)
            supportZone := box.new(left=bar_index - srLookback, top=support + tolerance, right=bar_index + 1, bottom=support - tolerance, border_color=color.new(color.lime, 20),
bgcolor=color.new(color.lime, 88), extend=extend.right, text="SUPPORT | zone possible de reaction", text_color=color.new(color.lime, 0), text_size=size.small, text_halign=text.align_left)
        else
            box.set_lefttop(supportZone, bar_index - srLookback, support + tolerance)
            box.set_rightbottom(supportZone, bar_index + 1, support - tolerance)
            box.set_extend(supportZone, extend.right)
    else if not na(supportZone)
        box.delete(supportZone)
        supportZone := na

    if showLevels and showZones and not na(resistance)
        if na(resistanceZone)
            resistanceZone := box.new(left=bar_index - srLookback, top=resistance + tolerance, right=bar_index + 1, bottom=resistance - tolerance, border_color=color.new(color.red, 20),
bgcolor=color.new(color.red, 88), extend=extend.right, text="RESISTANCE | zone possible de reaction", text_color=color.new(color.red, 0), text_size=size.small, text_halign=text.align_left)
        else
            box.set_lefttop(resistanceZone, bar_index - srLookback, resistance + tolerance)
            box.set_rightbottom(resistanceZone, bar_index + 1, resistance - tolerance)
            box.set_extend(resistanceZone, extend.right)
    else if not na(resistanceZone)
        box.delete(resistanceZone)
        resistanceZone := na

```

```
    if showLevels and showZones and armedDirection != 0 and not na(armedLevel)
        string retestText = armedDirection == 1 ? "RETEST ACHETEUR | attendre une cloture de rejet" : "RETEST VENDEUR | attendre une cloture de rejet"
        if na(retestZone)
            retestZone := box.new(left=armedBar, top=armedLevel + tolerance, right=bar_index + 1, bottom=armedLevel - tolerance, border_color=color.orange, bgcolor=color.new(color.orange, 82),
extend=extend.right, text=retestText, text_color=color.orange, text_size=size.small, text_halign=text.align_left)
        else
            box.set_lefttop(retestZone, armedBar, armedLevel + tolerance)
            box.set_rightbottom(retestZone, bar_index + 1, armedLevel - tolerance)
            box.set_text(retestZone, retestText)
            box.set_extend(retestZone, extend.right)
    else if not na(retestZone)
        box.delete(retestZone)
        retestZone := na

 var table status = table.new(position.top_right, 2, 11, border_width=1)
 if barstate.islast and showStatus
    string structureText = bullishStructure ? "HH/HL: sommets et creux ascendants" : bearishStructure ? "LH/LL: sommets et creux descendants" : "Structure mixte / sans sequence swing nette"
    string forecastSideText = forecastDirection == 1 ? "BIAIS HAUSSIER" : forecastDirection == -1 ? "BIAIS BAISSIER" : "BIAIS NEUTRE"
    string confluenceText = math.abs(forecastScore) >= 7 ? "confluence elevee" : math.abs(forecastScore) >= 5 ? "confluence moyenne" : math.abs(forecastScore) >= 3 ? "confluence faible" :
"aucune direction dominante"
    string forecastText = forecastSideText + " | score " + (forecastScore > 0 ? "+" : "") + str.tostring(forecastScore) + "/8 | " + confluenceText
    string rangePositionText = close > rangeMidpoint ? "prix > milieu" : close < rangeMidpoint ? "prix < milieu" : "au milieu"
    string scoreBreakdownText = "Structure " + (structureScore > 0 ? "+" : "") + str.tostring(structureScore) + " | Cassure/retest " + (breakoutScore > 0 ? "+" : "") +
str.tostring(breakoutScore) + " | Contexte " + (contextScore > 0 ? "+" : "") + str.tostring(contextScore) + " | Position " + rangePositionText + " " + (rangePositionScore > 0 ? "+" : "") +
str.tostring(rangePositionScore) + " | Momentum " + (horizonMomentumScore > 0 ? "+" : "") + str.tostring(horizonMomentumScore)
    bool showProDetails = panelMode == "Briefing pro"
    string contextText = not contextTimeframeOk ? "Regler sur une unite superieure a " + timeframe.period : bullishContext ? "HAUSSIER HH/HL | derniere bougie cloturee " +
str.tostring(contextClose, format.mintick) : bearishContext ? "BAISSIER LH/LL | derniere bougie cloturee " + str.tostring(contextClose, format.mintick) : "MIXTE | derniere cloture " +
str.tostring(contextClose, format.mintick)
    string structureStateText = bullishStructure ? "Tendance locale haussiere" : bearishStructure ? "Tendance locale baissiere" : not na(slowRange) and avgRange > slowRange * 1.35 ? "Expansion
/ volatilite elevee" : not na(slowRange) and avgRange < slowRange * 0.75 ? "Compression / attente de cassure" : "Range ou transition"
    bool forecastConflict = (forecastDirection == 1 and bearishContext) or (forecastDirection == -1 and bullishContext)
    string conflictText = forecastConflict ? " | ATTENTION: contre-biais unite superieure" : ""
    string horizonText = chartTimeframeOk ? str.tostring(forecastHorizonBars) + " bougies / ~" + str.tostring(forecastHorizonBars * timeframe.multiplier) + " min" :
str.tostring(forecastHorizonBars) + " bougies"
    string projectionText = forecastDirection == 1 ? horizonText + " | cible scenario " + str.tostring(forecastTarget, format.mintick) : forecastDirection == -1 ? horizonText + " | cible
scenario " + str.tostring(forecastTarget, format.mintick) : horizonText + " | pas de cible directionnelle"
    string invalidationText = forecastDirection == 1 ? "Biais invalide apres cloture sous " + str.tostring(forecastInvalidation, format.mintick) : forecastDirection == -1 ? "Biais invalide
apres cloture au-dessus de " + str.tostring(forecastInvalidation, format.mintick) : "Neutre: cloture > " + str.tostring(resistance + breakBuffer, format.mintick) + " ou < " + str.tostring(support -
breakBuffer, format.mintick)
    color forecastColor = forecastDirection == 1 ? color.new(color.green, 20) : forecastDirection == -1 ? color.new(color.red, 20) : color.new(color.gray, 45)
    string triggerText = armedDirection == 1 ? "Cassure haussiere armee: retest de " + str.tostring(armedLevel, format.mintick) + " puis rejet haussier" : armedDirection == -1 ? "Cassure
baissiere armee: retest de " + str.tostring(armedLevel, format.mintick) + " puis rejet baissier" : setupMode == "Cassure + retest" and forecastDirection == 1 ? "Long seulement apres cloture > " +
str.tostring(resistance + breakBuffer, format.mintick) + ", puis retest tenu" : setupMode == "Cassure + retest" and forecastDirection == -1 ? "Short seulement apres cloture < " +
str.tostring(support - breakBuffer, format.mintick) + ", puis retest tenu" : setupMode == "Cassure + retest" ? "Attendre cloture > " + str.tostring(resistance + breakBuffer, format.mintick) + " ou <
" + str.tostring(support - breakBuffer, format.mintick) : forecastDirection == 1 ? "Rejet haussier confirme dans la zone support " + str.tostring(support, format.mintick) : forecastDirection == -1 ?
"Rejet baissier confirme dans la zone resistance " + str.tostring(resistance, format.mintick) : "Attendre un rejet confirme sur support ou resistance"
    string stateText = "PAS DE SETUP | attendre, pas d'entree au milieu"
    if not chartTimeframeOk
        stateText := "TF non pris en charge: choisir M1 a M15"
    else if strategy.position_size > 0
        stateText := "POSITION LONGUE | gerer le plan"
    else if strategy.position_size < 0
        stateText := "POSITION SHORT | gerer le plan"
    else if armedDirection == 1
        stateText := "CASSURE HAUSSIERE | retest a attendre"
    else if armedDirection == -1
        stateText := "CASSURE BAISSIERE | retest a attendre"
    else if (confirmedLong and not longPlanValid) or (confirmedShort and not shortPlanValid)
        stateText := "Setup bloque: stop trop large ou frais eleves"
    else if (confirmedLong and not longQtyValid) or (confirmedShort and not shortQtyValid)
        stateText := "Setup bloque: taille minimale trop grande"
    else if placeLong
        stateText := "ACHAT PA confirme a la cloture"
```

```
     else if placeShort
         stateText := "VENTE PA confirmee a la cloture"
     string actionText = strategy.position_size != 0 ? "Position simulee ouverte; suivre stop et objectifs ci-dessous" : placeLong ? "Signal acheteur confirme a la cloture" : placeShort ? "Signal vendeur confirme a la cloture" : stateText
     string levelsText = str.tostring(support, format.mintick) + " | R: " + str.tostring(resistance, format.mintick)
     string tradeText = strategy.position_size > 0 ? "Stop " + str.tostring(activeStop, format.mintick) + " | TP1 " + str.tostring(longTp1Price, format.mintick) + " | TP2 " + str.tostring(longTp2Price, format.mintick) : strategy.position_size < 0 ? "Stop " + str.tostring(activeStop, format.mintick) + " | TP1 " + str.tostring(shortTp1Price, format.mintick) + " | TP2 " + str.tostring(shortTp2Price, format.mintick) : "Signal a la cloture; execution simulee au prochain tick"
     table.cell(status, 0, 0, "KRONOS PA | SCALP BRIEFING", text_color=color.white, bgcolor=color.new(color.blue, 15), text_size=size.small)
     table.cell(status, 1, 0, timeframe.period + " | " + setupMode, text_color=color.white, bgcolor=color.new(color.blue, 35), text_size=size.tiny)
     table.cell(status, 0, 1, "SCENARIO", text_color=color.white, bgcolor=color.new(color.gray, 35), text_size=size.tiny)
     table.cell(status, 1, 1, forecastText + conflictText, text_color=color.white, bgcolor=forecastColor, text_size=size.tiny)
     table.cell(status, 0, 2, showProDetails ? "STRUCTURE / REGIME" : "", text_color=color.white, bgcolor=showProDetails ? color.new(color.gray, 35) : color.new(color.gray, 100), text_size=size.tiny)
     table.cell(status, 1, 2, showProDetails ? structureText + " | " + structureStateText : "", text_color=color.white, bgcolor=showProDetails ? color.new(color.gray, 65) : color.new(color.gray, 100), text_size=size.tiny)
     table.cell(status, 0, 3, "CONTEXTE " + contextTimeframe, text_color=color.white, bgcolor=color.new(color.gray, 35), text_size=size.tiny)
     table.cell(status, 1, 3, contextText, text_color=color.white, bgcolor=bullishContext ? color.new(color.green, 55) : bearishContext ? color.new(color.red, 55) : color.new(color.gray, 65), text_size=size.tiny)
     table.cell(status, 0, 4, showProDetails ? "DETAIL DU SCORE" : "", text_color=color.white, bgcolor=showProDetails ? color.new(color.gray, 35) : color.new(color.gray, 100), text_size=size.tiny)
     table.cell(status, 1, 4, showProDetails ? scoreBreakdownText : "", text_color=color.white, bgcolor=showProDetails ? color.new(color.gray, 65) : color.new(color.gray, 100), text_size=size.tiny)
     table.cell(status, 0, 5, "HORIZON / CIBLE", text_color=color.white, bgcolor=color.new(color.gray, 35), text_size=size.tiny)
     table.cell(status, 1, 5, projectionText, text_color=color.white, bgcolor=color.new(color.blue, 65), text_size=size.tiny)
     table.cell(status, 0, 6, "INVALIDATION", text_color=color.white, bgcolor=color.new(color.gray, 35), text_size=size.tiny)
     table.cell(status, 1, 6, invalidationText, text_color=color.white, bgcolor=color.new(color.orange, 55), text_size=size.tiny)
     table.cell(status, 0, 7, showProDetails ? "SUPPORT / RESISTANCE" : "", text_color=color.white, bgcolor=showProDetails ? color.new(color.gray, 35) : color.new(color.gray, 100), text_size=size.tiny)
     table.cell(status, 1, 7, showProDetails ? "S: " + levelsText : "", text_color=color.white, bgcolor=showProDetails ? color.new(color.gray, 65) : color.new(color.gray, 100), text_size=size.tiny)
     table.cell(status, 0, 8, "DECLENCHEUR", text_color=color.white, bgcolor=color.new(color.gray, 35), text_size=size.tiny)
     table.cell(status, 1, 8, triggerText, text_color=color.white, bgcolor=color.new(color.blue, 65), text_size=size.tiny)
     table.cell(status, 0, 9, "ETAT", text_color=color.white, bgcolor=color.new(color.gray, 35), text_size=size.tiny)
     table.cell(status, 1, 9, actionText, text_color=color.white, bgcolor=strategy.position_size != 0 ? color.new(color.orange, 25) : color.new(color.gray, 65), text_size=size.tiny)
     table.cell(status, 0, 10, showProDetails ? "GESTION" : "", text_color=color.white, bgcolor=showProDetails ? color.new(color.gray, 35) : color.new(color.gray, 100), text_size=size.tiny)
     table.cell(status, 1, 10, showProDetails ? tradeText : "", text_color=color.white, bgcolor=showProDetails ? color.new(color.teal, 70) : color.new(color.gray, 100), text_size=size.tiny)
 if barstate.islast and not showStatus
     table.clear(status, 0, 0, 1, 10)

 if placeLong
     alert("KRONOS PA scalp BUY " + syminfo.ticker + " " + timeframe.period + " | setup=" + setupMode + " | close=" + str.tostring(close, format.mintick) + " | SL=" + str.tostring(longStop, format.mintick) + " | TP1=" + str.tostring(close + longRisk * tp1R, format.mintick) + " | TP2=" + str.tostring(close + longRisk * tp2R, format.mintick) + " | simulated entry=next tick", alert.freq_once_per_bar_close)
 if placeShort
     alert("KRONOS PA scalp SELL " + syminfo.ticker + " " + timeframe.period + " | setup=" + setupMode + " | close=" + str.tostring(close, format.mintick) + " | SL=" + str.tostring(shortStop, format.mintick) + " | TP1=" + str.tostring(close - shortRisk * tp1R, format.mintick) + " | TP2=" + str.tostring(close - shortRisk * tp2R, format.mintick) + " | simulated entry=next tick", alert.freq_once_per_bar_close)
```